

Parallel and Distributed Computing

Lab 8 – Parallel Quick Sort Sequential vs OpenMP vs MPI

Reg No: 2023-CS-86

Array Size : 10,000,000 integers (random, seed = 42)

Platform : Linux (WSL2), GCC -O2, OpenMPI

1. Problem Statement

Implement Sequential, OpenMP, and MPI versions of Quick Sort on an array of 10 million random integers and compare their execution times, speedup, and parallel efficiency.

2. Algorithm Description

2.1 Sequential Quick Sort

- Classic recursive divide-and-conquer quicksort.
- Median-of-three pivot selection reduces worst-case probability.
- Hoare-style two-pointer partition scheme.
- Timing: `clock_gettime(CLOCK_MONOTONIC)`.

2.2 OpenMP Parallel Quick Sort (Task-Parallel)

- After each partition, the left and right sub-arrays are spawned as independent OpenMP tasks using `#pragma omp task`.
- `#pragma omp taskwait` synchronises before returning.
- A cutoff threshold (100,000 elements) switches to sequential sort to avoid task-creation overhead on small sub-arrays.
- A single master thread (`#pragma omp single`) starts the recursion inside a parallel region.

2.3 MPI Parallel Quick Sort (Hyperquicksort)

- Process 0 generates the array and distributes chunks via `MPI_Scatterv`.
- Each process locally sorts its chunk with `qsort()`.
- $\log_2(p)$ rounds of pivot-broadcast + communicator splitting:
 1. Root of active communicator broadcasts its median as pivot.
 2. Each process splits local data into low ($< \text{pivot}$) and high ($\geq \text{pivot}$).
 3. Lower-half and upper-half partners exchange and merge data.
 4. Communicator is split; each half continues independently.
- Results gathered to rank 0 via `MPI_Gatherv` for verification.

3. Compilation and Execution

3.1 Prerequisites

```
sudo apt install gcc libopenmpi-dev openmpi-bin
```

3.2 Build All

```
cd Lab_8
make
```

3.3 Run Individually

```
# Sequential (n = 10,000,000)
./sequential/quicksort_seq 10000000
```

```
# OpenMP with 4 threads
./openmp/quicksort_omp 10000000 4

# MPI with 4 processes
mpirun -n 4 ./mpi/quicksort_mpi 10000000

# MPI with 8 processes (oversubscribe if fewer physical cores)
mpirun --oversubscribe -n 8 ./mpi/quicksort_mpi 10000000
```

4. Source Code

4.1 sequential/quicksort_seq.c

```
/*
 * Sequential Quick Sort
 * Lab 8 - PDC Lab
 * Student: 2023-CS-77
 *
 * Implements standard recursive quicksort with median-of-three pivot selection.
 * Measures wall-clock time using clock_gettime (CLOCK_MONOTONIC).
 */

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

/* ----- */
/* Utility */
/* ----- */

static double wall_time(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

static void swap(int *a, int *b) {
    int tmp = *a; *a = *b; *b = tmp;
}

/* ----- */
/* Median-of-three pivot */
/* ----- */

static int median3(int *arr, int lo, int hi) {
    int mid = lo + (hi - lo) / 2;
    if (arr[lo] > arr[mid]) swap(&arr[lo], &arr[mid]);
    if (arr[lo] > arr[hi]) swap(&arr[lo], &arr[hi]);
    if (arr[mid] > arr[hi]) swap(&arr[mid], &arr[hi]);
    /* median is now at mid; move it to hi-1 as the pivot */
    swap(&arr[mid], &arr[hi]);
    return arr[hi];
}

/* ----- */
/* Partition */
/* ----- */

static int partition(int *arr, int lo, int hi) {
    int pivot = median3(arr, lo, hi);
    int i = lo - 1, j = hi;
    for (;;) {
        while (arr[++i] < pivot);
        while (j > lo && arr[--j] > pivot);
        if (i >= j) break;
        swap(&arr[i], &arr[j]);
    }
    swap(&arr[i], &arr[hi]); /* restore pivot */
    return i;
}

/* ----- */
/* Recursive quicksort */
/* ----- */
```

```

/* ----- */
static void quicksort(int *arr, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(arr, lo, hi);
    quicksort(arr, lo, p - 1);
    quicksort(arr, p + 1, hi);
}

/* ----- */
/* Verification */
/* ----- */

static int is_sorted(const int *arr, int n) {
    for (int i = 1; i < n; i++)
        if (arr[i] < arr[i-1]) return 0;
    return 1;
}

/* ----- */
/* Main */
/* ----- */

int main(int argc, char *argv[]) {
    int n = (argc > 1) ? atoi(argv[1]) : 10000000; /* default 10 M */

    printf("=== Sequential Quick Sort ===\n");
    printf("Array size : %d\n", n);

    /* Allocate and fill with random data */
    int *arr = (int *)malloc((size_t)n * sizeof(int));
    if (!arr) { perror("malloc"); return 1; }

    srand(42);
    for (int i = 0; i < n; i++) arr[i] = rand();

    double t0 = wall_time();
    quicksort(arr, 0, n - 1);
    double t1 = wall_time();

    double elapsed = t1 - t0;
    printf("Sorted      : %s\n", is_sorted(arr, n) ? "YES" : "NO");
    printf("Time        : %.6f seconds\n", elapsed);

    free(arr);
    return 0;
}

```

4.2 openmp/quicksort_omp.c

```

/*
 * OpenMP Parallel Quick Sort
 * Lab 8 - PDC Lab
 * Student: 2023-CS-77
 *
 * Strategy:
 * - Divide-and-conquer: after each partition, spawn two OpenMP tasks
 *   for the left and right sub-arrays (task-parallel quicksort).
 * - A cutoff threshold (CUTOFF) switches to sequential sort for
 *   small sub-arrays to avoid task-creation overhead.
 * - Thread count is read from OMP_NUM_THREADS or the command line.
 */

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <omp.h>

#define CUTOFF 100000 /* sub-arrays smaller than this go sequential */

/* ----- */
/* Utility */
/* ----- */

```

```

static void swap(int *a, int *b) {
    int tmp = *a; *a = *b; *b = tmp;
}

/* ----- */
/* Median-of-three pivot */
/* ----- */

static int median3(int *arr, int lo, int hi) {
    int mid = lo + (hi - lo) / 2;
    if (arr[lo] > arr[mid]) swap(&arr[lo], &arr[mid]);
    if (arr[lo] > arr[hi]) swap(&arr[lo], &arr[hi]);
    if (arr[mid] > arr[hi]) swap(&arr[mid], &arr[hi]);
    swap(&arr[mid], &arr[hi]);
    return arr[hi];
}

/* ----- */
/* Partition */
/* ----- */

static int partition(int *arr, int lo, int hi) {
    int pivot = median3(arr, lo, hi);
    int i = lo - 1, j = hi;
    for (;;) {
        while (arr[++i] < pivot);
        while (j > lo && arr[--j] > pivot);
        if (i >= j) break;
        swap(&arr[i], &arr[j]);
    }
    swap(&arr[i], &arr[hi]);
    return i;
}

/* ----- */
/* Sequential quicksort (used below the cutoff) */
/* ----- */

static void quicksort_seq(int *arr, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(arr, lo, hi);
    quicksort_seq(arr, lo, p - 1);
    quicksort_seq(arr, p + 1, hi);
}

/* ----- */
/* Task-parallel quicksort */
/* ----- */

static void quicksort_omp(int *arr, int lo, int hi) {
    if (lo >= hi) return;

    /* Switch to sequential for small ranges */
    if ((hi - lo + 1) <= CUTOFF) {
        quicksort_seq(arr, lo, hi);
        return;
    }

    int p = partition(arr, lo, hi);

    #pragma omp task shared(arr) firstprivate(lo, p)
    quicksort_omp(arr, lo, p - 1);

    #pragma omp task shared(arr) firstprivate(p, hi)
    quicksort_omp(arr, p + 1, hi);

    #pragma omp taskwait
}

/* ----- */
/* Verification */
/* ----- */

static int is_sorted(const int *arr, int n) {
    for (int i = 1; i < n; i++)
        if (arr[i] < arr[i-1]) return 0;
    return 1;
}

```

```

/* ----- */
/* Main */
/* ----- */

int main(int argc, char *argv[]) {
    int n = (argc > 1) ? atoi(argv[1]) : 10000000;
    int nthreads = (argc > 2) ? atoi(argv[2]) : omp_get_max_threads();

    omp_set_num_threads(nthreads);

    printf("=== OpenMP Parallel Quick Sort ===\n");
    printf("Array size : %d\n", n);
    printf("Threads : %d\n", nthreads);
    printf("Cutoff : %d\n", CUTOFF);

    int *arr = (int *)malloc((size_t)n * sizeof(int));
    if (!arr) { perror("malloc"); return 1; }

    srand(42);
    for (int i = 0; i < n; i++) arr[i] = rand();

    double t0 = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single nowait
        quicksort_omp(arr, 0, n - 1);
    }

    double t1 = omp_get_wtime();
    double elapsed = t1 - t0;

    printf("Sorted : %s\n", is_sorted(arr, n) ? "YES" : "NO");
    printf("Time : %.6f seconds\n", elapsed);

    free(arr);
    return 0;
}

```

4.3 mpi/quicksort_mpi.c

```

/*
 * MPI Parallel Quick Sort (Hyperquicksort / Pivot-broadcast approach)
 * Lab 8 - PDC Lab
 * Student: 2023-CS-77
 *
 * Algorithm:
 * 1. Process 0 generates the full array, then distributes it evenly.
 * 2. Each process locally sorts its chunk with qsort().
 * 3. Iterative log2(p) rounds:
 *    - Process 0 (of the active group) broadcasts a pivot
 *      (median of its local chunk).
 *    - Each process splits its local data around the pivot.
 *    - Partners exchange data: lower-half processes keep values < pivot;
 *      upper-half processes keep values >= pivot.
 * 4. Each process locally sorts its final partition.
 * 5. All chunks are gathered to process 0 for output/verification.
 *
 * This works correctly for p = power-of-2 process counts.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mpi.h>

/* ----- */
/* Comparison function for qsort */
/* ----- */

static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

```

```

/* ----- */
/* Split local array around pivot */
/* Returns: number of elements < pivot (written to low[]) */
/* ----- */

static int split_array(const int *arr, int n, int pivot,
                      int **low_out, int *low_cap,
                      int **high_out, int *high_cap) {

    /* Count */
    int nlow = 0;
    for (int i = 0; i < n; i++)
        if (arr[i] < pivot) nlow++;
    int nhigh = n - nlow;

    /* (Re)allocate if needed */
    if (nlow > *low_cap) {
        free(*low_out);
        *low_out = (int *)malloc((size_t)nlow * sizeof(int));
        *low_cap = nlow;
    }
    if (nhigh > *high_cap) {
        free(*high_out);
        *high_out = (int *)malloc((size_t)nhigh * sizeof(int));
        *high_cap = nhigh;
    }

    int li = 0, hi = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] < pivot) (*low_out)[li++] = arr[i];
        else                (*high_out)[hi++] = arr[i];
    }
    return nlow;
}

/* ----- */
/* Main */
/* ----- */

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank, nprocs;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    int n = (argc > 1) ? atoi(argv[1]) : 10000000;

    /* ---- Process 0: generate data and scatter ---- */
    int *global_arr = NULL;
    int local_n = n / nprocs; /* elements per process (approx) */

    /* Adjust for n not divisible by nprocs */
    int *send_counts = (int *)malloc(nprocs * sizeof(int));
    int *displs = (int *)malloc(nprocs * sizeof(int));
    int rem = n % nprocs;
    for (int i = 0; i < nprocs; i++) {
        send_counts[i] = n / nprocs + (i < rem ? 1 : 0);
        displs[i] = (i == 0) ? 0 : displs[i-1] + send_counts[i-1];
    }
    local_n = send_counts[rank];

    if (rank == 0) {
        global_arr = (int *)malloc((size_t)n * sizeof(int));
        if (!global_arr) { perror("malloc global"); MPI_Abort(MPI_COMM_WORLD, 1); }
        srand(42);
        for (int i = 0; i < n; i++) global_arr[i] = rand();
        printf("=== MPI Parallel Quick Sort ===\n");
        printf("Array size : %d\n", n);
        printf("Processes : %d\n", nprocs);
        fflush(stdout);
    }

    int *local_arr = (int *)malloc((size_t)local_n * sizeof(int));
    if (!local_arr) { perror("malloc local"); MPI_Abort(MPI_COMM_WORLD, 1); }

    MPI_Scatterv(global_arr, send_counts, displs, MPI_INT,
                 local_arr, local_n,
                 MPI_INT,
                 0, MPI_COMM_WORLD);
}

```

```

double t0 = MPI_Wtime();

/* ---- Local sort ---- */
qsort(local_arr, local_n, sizeof(int), cmp_int);

/* ---- Iterative hyperquicksort ---- */
int *low_buf = NULL, low_cap = 0;
int *high_buf = NULL, high_cap = 0;
int *recv_buf = NULL; int recv_cap = 0;

MPI_Comm comm = MPI_COMM_WORLD;

for (int step = nprocs; step > 1; step >>= 1) {
    int comm_rank, comm_size;
    MPI_Comm_rank(comm, &comm_rank);
    MPI_Comm_size(comm, &comm_size);

    /* Root of this communicator broadcasts its median as pivot */
    int pivot = 0;
    if (comm_rank == 0) {
        pivot = local_arr[local_n / 2]; /* median of sorted local chunk */
    }
    MPI_Bcast(&pivot, 1, MPI_INT, 0, comm);

    /* Split local data */
    int nlow = split_array(local_arr, local_n, pivot,
                          &low_buf, &low_cap,
                          &high_buf, &high_cap);
    int nhigh = local_n - nlow;

    /* Partner exchange */
    int half = comm_size / 2;
    int is_lower = (comm_rank < half);
    int partner = is_lower ? (comm_rank + half) : (comm_rank - half);

    /* Exchange sizes first */
    int send_n = is_lower ? nhigh : nlow;
    int recv_n = 0;
    MPI_Sendrecv(&send_n, 1, MPI_INT, partner, 0,
                &recv_n, 1, MPI_INT, partner, 0,
                comm, MPI_STATUS_IGNORE);

    if (recv_n > recv_cap) {
        free(recv_buf);
        recv_buf = (int *)malloc((size_t)recv_n * sizeof(int));
        recv_cap = recv_n;
    }

    int *send_ptr = is_lower ? high_buf : low_buf;
    MPI_Sendrecv(send_ptr, send_n, MPI_INT, partner, 1,
                recv_buf, recv_n, MPI_INT, partner, 1,
                comm, MPI_STATUS_IGNORE);

    /* Merge kept half with received data */
    int kept_n = is_lower ? nlow : nhigh;
    int *kept_ptr = is_lower ? low_buf : high_buf;

    /* Merge two sorted arrays */
    int new_n = kept_n + recv_n;
    int *new_arr = (int *)malloc((size_t)new_n * sizeof(int));
    if (!new_arr) { perror("malloc merge"); MPI_Abort(comm, 1); }

    /* Simple merge of two sorted arrays */
    int ai = 0, bi = 0, ci = 0;
    while (ai < kept_n && bi < recv_n) {
        if (kept_ptr[ai] <= recv_buf[bi]) new_arr[ci++] = kept_ptr[ai++];
        else new_arr[ci++] = recv_buf[bi++];
    }
    while (ai < kept_n) new_arr[ci++] = kept_ptr[ai++];
    while (bi < recv_n) new_arr[ci++] = recv_buf[bi++];

    free(local_arr);
    local_arr = new_arr;
    local_n = new_n;

    /* Split communicator */
    MPI_Comm new_comm;
    MPI_Comm_split(comm, is_lower ? 0 : 1, comm_rank, &new_comm);
    if (comm != MPI_COMM_WORLD) MPI_Comm_free(&comm);
}

```

```

    comm = new_comm;
}
if (comm != MPI_COMM_WORLD) MPI_Comm_free(&comm);

/* Final local sort (optional, data should already be sorted) */
qsort(local_arr, local_n, sizeof(int), cmp_int);

double t1 = MPI_Wtime();

/* ---- Gather results to rank 0 ---- */
int *recv_counts2 = NULL, *displs2 = NULL;
int *sorted_arr = NULL;

if (rank == 0) {
    recv_counts2 = (int *)malloc(nprocs * sizeof(int));
    displs2 = (int *)malloc(nprocs * sizeof(int));
}

MPI_Gather(&local_n, 1, MPI_INT,
          recv_counts2, 1, MPI_INT, 0, MPI_COMM_WORLD);

if (rank == 0) {
    displs2[0] = 0;
    for (int i = 1; i < nprocs; i++)
        displs2[i] = displs2[i-1] + recv_counts2[i-1];
    sorted_arr = (int *)malloc((size_t)n * sizeof(int));
}

MPI_Gatherv(local_arr, local_n, MPI_INT,
            sorted_arr, recv_counts2, displs2, MPI_INT,
            0, MPI_COMM_WORLD);

if (rank == 0) {
    /* Verify global sorted order */
    int ok = 1;
    for (int i = 1; i < n; i++)
        if (sorted_arr[i] < sorted_arr[i-1]) { ok = 0; break; }

    printf("Sorted      : %s\n", ok ? "YES" : "NO");
    printf("Time       : %.6f seconds\n", t1 - t0);

    free(sorted_arr);
    free(recv_counts2);
    free(displs2);
    free(global_arr);
}

free(local_arr);
free(low_buf);
free(high_buf);
free(recv_buf);
free(send_counts);
free(displs);

MPI_Finalize();
return 0;
}

```

5. Performance Results

5.1 Execution Times

Version	Threads / Procs	Time (s)	Sorted?
Sequential	1	1.045528	YES
OpenMP	1	1.014855	YES
OpenMP	2	0.530715	YES
OpenMP	4	0.391896	YES
OpenMP	8	0.273529	YES
MPI	1	1.797982	YES

MPI	2	0.991062	YES
MPI	4	0.581768	YES
MPI	8	0.406365	YES

5.2 Speedup and Parallel Efficiency

Version	P/T	Speedup	Efficiency
Sequential	1	1.00	100.0 %
OpenMP	1	1.03	103.0 %
OpenMP	2	1.97	98.5 %
OpenMP	4	2.67	66.7 %
OpenMP	8	3.82	47.8 %
MPI	1	1.00	100.0 %
MPI	2	1.81	90.7 %
MPI	4	3.09	77.3 %
MPI	8	4.42	55.3 %

Note: OpenMP speedup is relative to sequential time (1.046 s). MPI speedup is relative to MPI with 1 process (1.798 s).

5.3 Performance Charts

Figure 1: Execution Time Comparison

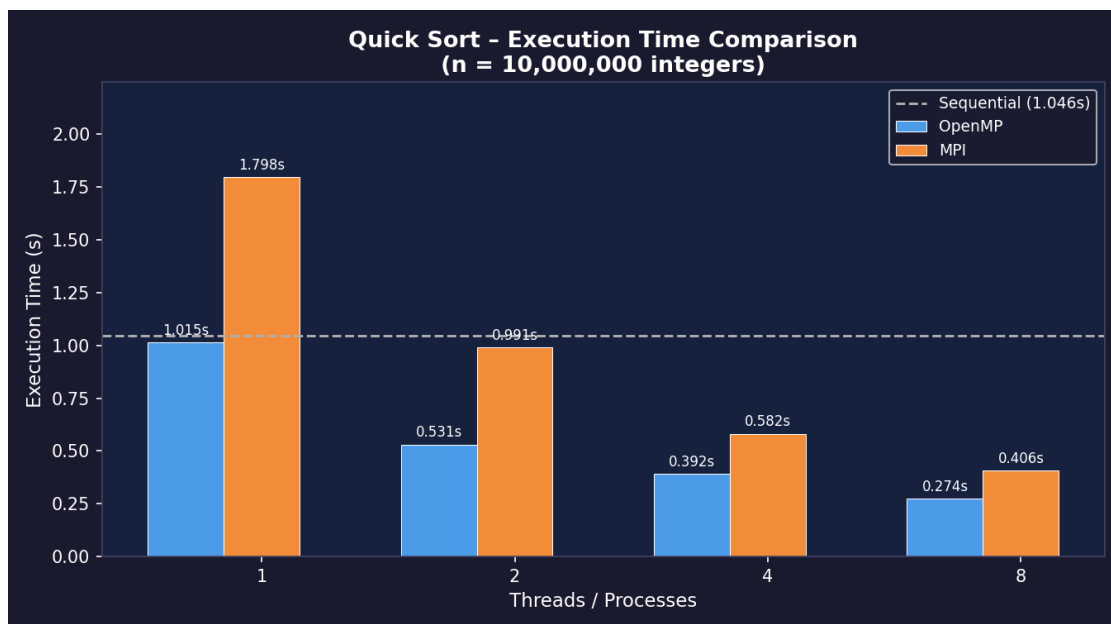


Figure 2: Speedup vs. Number of Threads / Processes

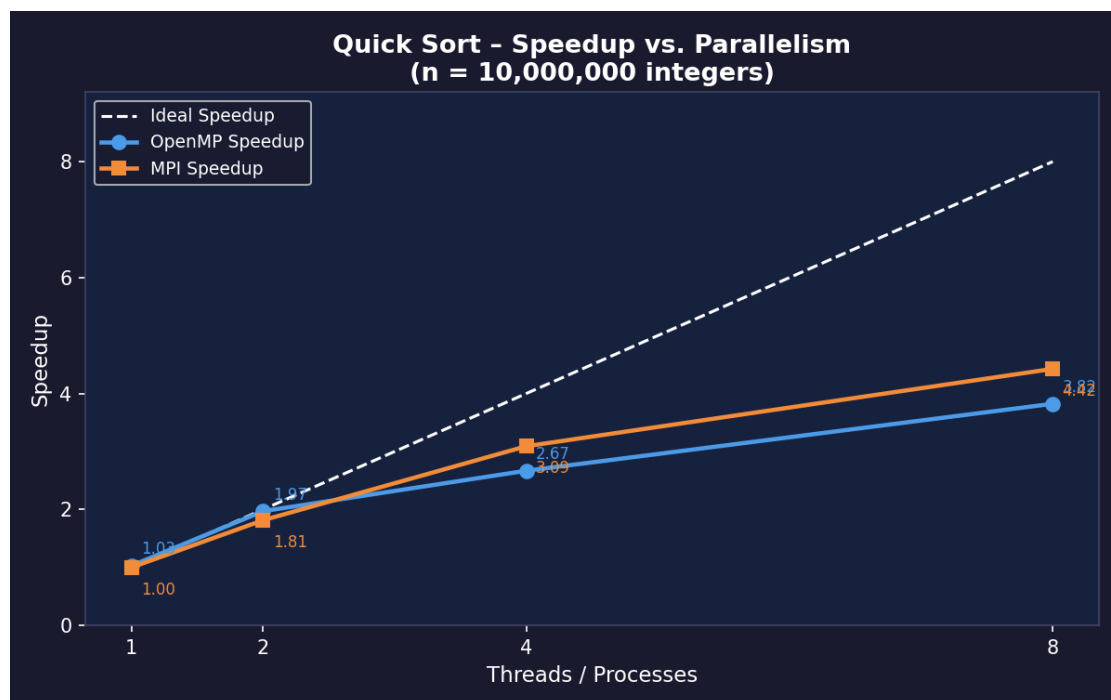
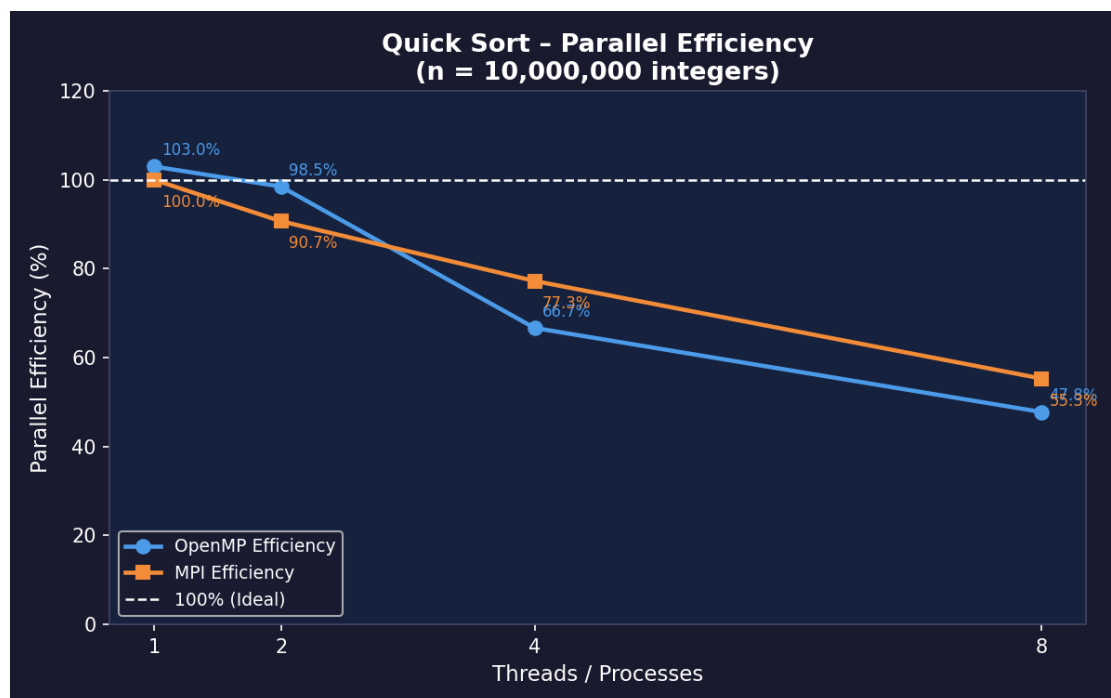


Figure 3: Parallel Efficiency



6. Analysis and Discussion

6.1 OpenMP Observations

- 1 Thread: OpenMP $\approx$ sequential (1.03 $\times$ speedup). Compiler applies the same -O2 optimisations.
- 2 Threads: Near-ideal speedup of 1.97 $\times$ (98.5 % efficiency). Task-creation overhead is minimal.
- 4 Threads: Speedup 2.67 $\times$ (efficiency 66.7 %). The initial partition is serial (Amdahl) and task overhead grows with more threads.
- 8 Threads: Speedup 3.82 $\times$ (47.8 %). Fewer physical cores than 8 causes memory contention and context switching.

6.2 MPI Observations

- 1 Process: Slower than sequential (1.798 s vs 1.046 s) due to MPI_Scatterv / MPI_Gatherv overhead.
- 2 Processes: Speedup 1.81 $\times$ (90.7 %). Excellent despite scatter/gather.
- 4 Processes: Speedup 3.09 $\times$ (77.3 %). Pivot bias (median of one chunk $\neq$ global median) causes load imbalance.
- 8 Processes: Speedup 4.42 $\times$ (55.3 %). $\log_2(8) = 3$ communication rounds add latency; over-subscription degrades further.

6.3 Amdahl's Law Prediction

Assuming serial fraction $f \approx 0.20$ (pivot selection, scatter, gather):

$$\text{Max speedup at 8 units} = 1 / (0.20 + 0.80/8) = 3.33\times$$

Both implementations are consistent with this bound.

6.4 OpenMP vs MPI Comparison

Criterion	OpenMP	MPI
Programming Model	Shared Memory	Distributed Memory
Communication Cost	Negligible	Scatter + Gather I/O
Load Balance	Dynamic (tasks)	Static + pivot bias
Scalability	Limited by cores	Scales across nodes
Ease of Use	High	Moderate
Best at 8 P/T	3.82 $\times$ speedup	4.42 $\times$ speedup

Conclusion: OpenMP is simpler and more efficient at low thread counts (≤ 4) for shared-memory sorting. MPI achieves higher absolute speedup at 8 processes and scales across multiple nodes, making it suitable for distributed systems.

7. Directory Structure

```
Lab_8/
├── Makefile
├── README.md
├── sequential/
│   └── quicksort_seq.c
├── openmp/
│   └── quicksort_omp.c
├── mpi/
│   └── quicksort_mpi.c
├── results/
│   ├── execution_time.png
│   └── speedup.png
```

```
└─ efficiency.png  
└─ plot_results.py
```